

Surviving Failure

The rest of Part VI — the patterns that keep a system alive when things break.

36 Timeouts, Retries & Backoff — bounding waits and recovering safely

37 Circuit Breakers & Bulkheads — stopping and containing failure

38 Sagas & Distributed Consistency — cross-service transactions without 2PC

39 Bloom Filters & Probabilistic Data Structures — approximate answers, tiny memory

40 Graceful Degradation & Chaos Engineering — degrade, don't collapse; test failure

These five patterns complete Part VI: bound failures, contain them, keep data consistent, save memory, and prove resilience by breaking things on purpose.

CHAPTER 36

Timeouts, Retries & Backoff

Part VI · Reliability & Resilience — bounding waits and recovering safely.

LEARNING OBJECTIVES

- Explain why every remote call needs a timeout.
- Explain when retries help and why only idempotent calls may be retried.
- Explain exponential backoff and why jitter is essential.
- Explain how retries can amplify an outage and how to prevent it.

REAL-WORLD ANALOGY — KNOCKING ON A DOOR

You knock and wait a *reasonable* time, not forever (a **timeout**). If no answer, you knock again — but after a pause, and only a few times (**retries with backoff**). And if a whole crowd is knocking on the same door, you don't all knock in perfect unison, or you'll just make a racket (**jitter**). These three habits keep a caller sane when the other side is slow or down.

36.1 The problem

Remote calls fail and hang (Chapter 19). Without limits, a slow dependency makes callers wait indefinitely, threads pile up, and the caller itself hangs — a **cascading failure**. Three tools contain this: **timeouts** bound waits, **retries** recover from transient blips, and **backoff + jitter** keep retries from making things worse.

36.2 Timeouts

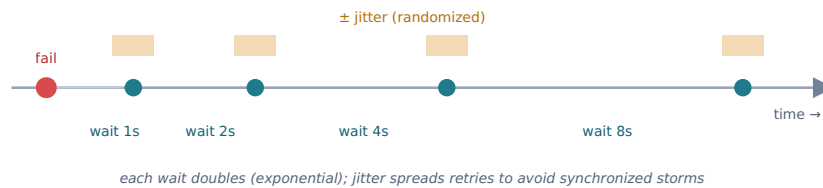
Never wait indefinitely — put a timeout on **every** remote call. Too long ties up threads; too short causes false failures. Work within a **latency budget** (Chapter 5): the overall request has a deadline, and each sub-call must fit inside it, ideally by **propagating the deadline** down the call chain so nobody works on a request that has already given up.

36.3 Retries

ONLY RETRY IDEMPOTENT OPERATIONS

Many failures are **transient** (a dropped packet, a brief overload) and succeed on a second attempt. But retrying is only safe for **idempotent** operations (Chapters 4, 25) — retrying a bare **POST** after a timeout can charge a card twice. Cap the retry count, and never retry a non-idempotent call blindly.

36.4 Backoff and jitter



Exponential backoff waits progressively longer between retries (1s, 2s, 4s, 8s...), giving a struggling dependency time to recover. And **jitter** — adding randomness to each wait — is essential: without it, many clients back off on the same schedule and retry in synchronized waves (a thundering herd, Chapter 8), hammering the service at the exact same moments.

36.5 Retries can amplify outages

KEY INSIGHT — RETRIES ADD LOAD EXACTLY WHEN YOU CAN LEAST AFFORD IT

A struggling service receives *extra* traffic from everyone's retries — a **retry storm** that turns a brief hiccup into a full outage. Defend against it: combine retries with a **circuit breaker** (Chapter 37), enforce a **retry budget** (cap total retries system-wide), and only retry when it can actually help. Retries are a scalpel, not a reflex.

Common mistakes

WATCH OUT

- Any remote call with no timeout — the classic cause of hung threads.
- Retrying non-idempotent operations and causing duplicates.
- Immediate or fixed-interval retries with no backoff.
- Backoff **without jitter**, producing synchronized retry waves.
- Unbounded retries that amplify an outage into a retry storm.

Interview questions

1. Why does every remote call need a timeout?
2. When are retries safe, and why only for idempotent operations?
3. What is exponential backoff, and why add jitter?
4. How can retries make an outage worse, and how do you prevent it?

Hands-on exercises

1. Add a connection and read timeout to a remote call and choose sensible values.
2. Mark which of GET, POST, PUT, DELETE you'd retry, and why.
3. Compute the retry times for exponential backoff over five attempts.
4. Explain how jitter prevents synchronized retries across 1,000 clients.

Mini project

BUILD A RESILIENT CLIENT

Wrap a call to a flaky service with a timeout, capped retries, exponential backoff, and jitter. Simulate a brief outage and confirm the client recovers. Then remove jitter and launch many clients at once to observe synchronized retry waves — then add jitter back and watch them spread out.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — CALL POLICIES

For each network boundary in your failure inventory (Chapter 19), set a **timeout**, decide whether the call is **retryable** (idempotent), and define **backoff + jitter** and a retry cap. Propagate a request **deadline** so downstream calls don't work on already-abandoned requests. Note where a retry budget protects a struggling dependency.

Chapter summary

- Put a **timeout** on every remote call and work within a propagated **deadline**.
- **Retry** only idempotent operations, with a capped count.
- Use **exponential backoff** plus **jitter** to avoid synchronized retry storms.
- Retries **amplify load** — pair them with circuit breakers and retry budgets.

CHAPTER 37

Circuit Breakers & Bulkheads

Part VI · Reliability & Resilience — stopping and containing failure.

LEARNING OBJECTIVES

- Explain how a failing dependency causes cascading failure.
- Describe the circuit breaker's three states and transitions.
- Explain the bulkhead pattern and how it limits blast radius.
- Explain fail-fast and fallbacks.

REAL-WORLD ANALOGY — BREAKERS AND SHIP COMPARTMENTS

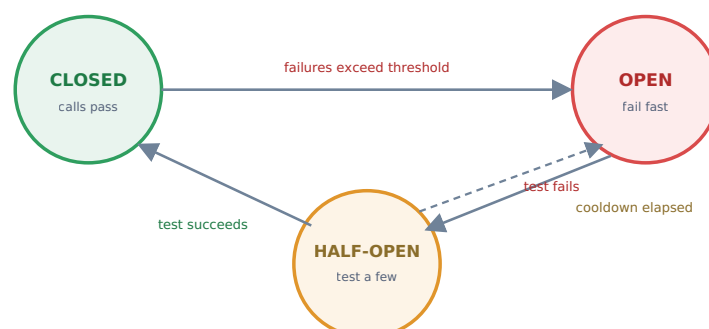
A home's electrical **circuit breaker** trips when a fault appears, cutting the circuit so the fault can't burn down the house. A ship's **bulkheads** seal compartments so that one flooded section doesn't sink the whole vessel. Both are failure *containment*: stop the bad thing, and limit how far it spreads.

37.1 The problem

When a dependency is down, continuing to call it — and retry it (Chapter 36) — wastes resources, ties up threads waiting on timeouts, and pushes the failure **upstream** to callers, and their callers. One broken service can take down an entire system. We need to **stop calling** a broken dependency and **contain** the damage.

37.2 The circuit breaker

A **circuit breaker** wraps calls to a dependency and watches their success/failure. It has three states:



Closed: calls pass through; failures are counted. **Open:** once failures cross a threshold, the breaker “trips” and **fails fast** — returning an error or fallback immediately without calling the dependency, giving it time to recover. **Half-open:** after a cooldown, a few test calls are allowed; success closes the breaker, failure re-opens it.

37.3 Bulkheads

The **bulkhead** pattern isolates resources so one failing part can't consume everything. For example, give each dependency its **own thread pool or connection pool**: a slow dependency can exhaust only *its* pool, not all of the application's threads. This limits the **blast radius** — one drowning compartment doesn't sink the ship.

37.4 Fail fast and fallbacks

KEY INSIGHT — FAILING FAST BEATS FAILING SLOW

A call that fails *immediately* is far better than one that hangs until a timeout — it frees threads and stops the failure spreading. And when the breaker is open, returning a **fallback** (cached data, a sensible default, a degraded response) instead of an error keeps users served. Circuit breakers stop calling broken things; bulkheads cap what any one thing can consume; fallbacks keep the experience alive (graceful degradation, Chapter 40). Libraries like resilience4j and proxies like Envoy provide these.

Common mistakes

WATCH OUT

- Retrying a dead dependency forever with no breaker to stop the bleeding.
- One shared thread pool for all dependencies — one slow call starves everything.
- No fallback when the breaker opens, so users see hard errors.
- Thresholds/cooldowns so sensitive they flap (open/close repeatedly).

Interview questions

1. How does a failing dependency cause cascading failure?
2. Describe the circuit breaker's three states and transitions.
3. What is the bulkhead pattern, and how does it limit blast radius?
4. Why is failing fast better than failing slow?
5. What role do fallbacks play when a breaker is open?

Hands-on exercises

1. Draw the circuit-breaker state machine and label each transition.
2. Explain how a per-dependency thread pool contains a slow service.
3. Design a fallback for a failed "recommendations" service.
4. Choose failure threshold and cooldown values and justify them.

Mini project

WRAP A DEPENDENCY IN A BREAKER

Put a circuit breaker around a call to a flaky service (using a library or a small implementation). Drive failures until it opens and confirm calls then fail fast; wait the

cooldown and watch it go half-open and recover. Add a fallback (cached value) for the open state, and give the dependency its own bounded thread pool to prove the bulkhead.

| Large project (capstone continues)

YOUR SOCIAL PLATFORM — CONTAIN THE FAILURES

Wrap calls to non-critical dependencies (recommendations, trending, avatar service) in **circuit breakers** with sensible thresholds and **fallbacks** (cached or default). Give each external dependency its own **bulkhead** (isolated pool). Confirm that a failure in any one leaves the core feed and posting paths working.

| Chapter summary

- A failing dependency plus retries causes **cascading failure**; breakers and bulkheads contain it.
- A **circuit breaker** moves through **Closed → Open → Half-Open**, failing fast while a dependency recovers.
- **Bulkheads** isolate resources (separate pools) to limit blast radius.
- **Fail fast** and return **fallbacks** to keep the system responsive.

CHAPTER 38

Sagas & Distributed Consistency

Part VI · Reliability & Resilience — cross-service transactions without 2PC.

LEARNING OBJECTIVES

- Explain why microservices can't use one ACID transaction across services.
- Describe the Saga pattern and compensating transactions.
- Compare choreographed and orchestrated sagas.
- Know the challenges: no isolation, failing compensations.

REAL-WORLD ANALOGY — A MULTI-LEG TRIP

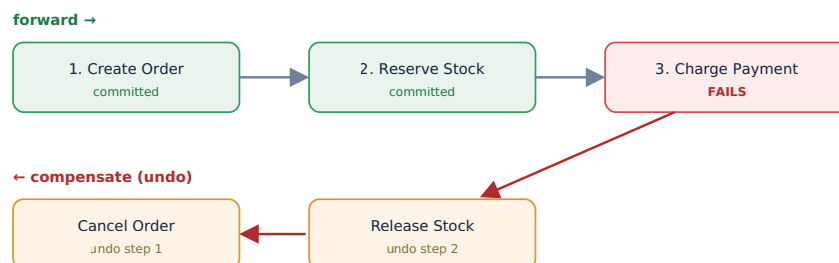
Booking a trip means confirming a flight, then a hotel, then a car — each a separate, already-final booking. If the car fails, you can't magically un-happen the flight and hotel; you must actively **cancel** them (a compensation). A **Saga** is exactly this: a chain of committed local steps, each with an undo action if a later step fails.

38.1 The problem

Chapter 24 showed distributed transactions (2PC) are blocking and don't scale, and Chapter 29 gave each microservice its own database. So a business operation spanning services **can't be one ACID transaction**. How do we keep data consistent across services without 2PC? The answer is the **Saga**.

38.2 What a Saga is

A Saga breaks a distributed transaction into a **sequence of local transactions**, each committed in one service. If any step fails, the Saga runs **compensating transactions** to undo the previously-completed steps. The result is **eventually consistent** rather than atomic — but available and scalable.



38.3 Compensating transactions

Each step needs a **compensation** that semantically undoes it — not a database rollback (the local transaction already committed), but a new action: “reserve stock” is undone by

“release stock,” “charge payment” by “refund.” Compensations should be **idempotent** and retryable, since they too run over an unreliable network.

38.4 Choreography vs orchestration

Choreographed saga	Orchestrated saga
Each service listens for events and emits the next; failures trigger compensating events. Decentralized, but the flow is implicit.	A central orchestrator drives each step and issues compensations on failure. Easier to understand and debug for complex sagas.

38.5 Challenges

SAGAS TRADE ATOMICITY FOR AVAILABILITY

Unlike an ACID transaction, a Saga has **no isolation** — intermediate states are visible (an order briefly exists before payment), so you may need **semantic locks** or careful state design. Compensations can themselves **fail** and must be retried, and the overall flow is more complex with an eventual-consistency window. You accept these to avoid the blocking and coupling of 2PC.

Common mistakes

WATCH OUT

- Forgetting a compensation for a step, leaving orphaned state on failure.
- Non-idempotent compensations that break when retried.
- Ignoring visible intermediate states (no semantic locking where needed).
- Choreographing a complex saga no one can trace end to end.

Interview questions

1. Why can't a microservices operation be a single ACID transaction?
2. What is a Saga, and how do compensating transactions work?
3. Compare choreographed and orchestrated sagas.
4. What consistency guarantees does a Saga give up, and why is that acceptable?

Hands-on exercises

1. Write the steps and compensations for a “place order” saga across order, inventory, and payment services.
2. Explain why a compensation is not the same as a database rollback.
3. Decide choreography vs orchestration for a 5-step saga and justify.
4. Describe an intermediate state a user might briefly see, and how to handle it.

Mini project

IMPLEMENT AN ORCHESTRATED SAGA

Model an order across three services (order, inventory, payment) with a small orchestrator. Run the happy path, then force the payment step to fail and watch the orchestrator issue compensations (release stock, cancel order). Make each step and compensation idempotent and retry a failed compensation to prove robustness.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE SAGAS

Turn the cross-service operations you listed in Chapter 24 into concrete sagas — e.g. sign-up (create user → provision feed → send welcome), each step with a compensation. Choose orchestration for complex flows and choreography for simple event chains, and specify idempotent compensations and how you'll handle visible intermediate states.

Chapter summary

- Microservices can't share one ACID transaction; a **Saga** is a chain of local transactions with **compensations**.
- Compensations **semantically undo** committed steps (release, refund) and must be idempotent.
- **Choreography** (event-driven) vs **orchestration** (central coordinator) — a clarity trade-off.
- Sagas are eventually consistent with **no isolation** — the accepted price for avoiding 2PC.

CHAPTER 39

Bloom Filters & Probabilistic Data Structures

Part VI · Reliability & Resilience — approximate answers in tiny memory.

LEARNING OBJECTIVES

- Explain why approximate structures save enormous memory.
- Explain how a Bloom filter tests membership and why it has false positives but no false negatives.
- List real uses of Bloom filters.
- Know HyperLogLog and Count-Min Sketch at a high level.

REAL-WORLD ANALOGY — A BOUNCER’S QUICK CHECK

A bouncer with a rough mental list can instantly say “**definitely not** on the guest list” or “**probably** on it — go check.” The quick check is tiny and fast; occasionally it waves through someone who isn’t actually listed, but it never wrongly turns away a real guest. A **Bloom filter** trades that small, one-sided error for enormous space savings.

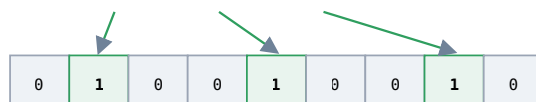
39.1 The problem

Sometimes you must test membership, count uniques, or measure frequencies over **huge** datasets, where exact structures (sets, hash tables) use too much memory. **Probabilistic data structures** give **approximate** answers in a tiny, fixed amount of memory, accepting a small, bounded error.

39.2 The Bloom filter

A **Bloom filter** tests set membership using a bit array and k hash functions. To **add** an item, hash it k ways and set those k bits to 1. To **check** an item, hash it and test those bits: if **any** is 0 it is **definitely not** in the set; if **all** are 1 it is **probably** in the set.

add x: hash 3 ways → set 3 bits



check y → any bit 0?
definitely NOT present

check z → all bits 1?
probably present

no false negatives; a small, tunable false-positive rate — and huge memory savings

So a Bloom filter can have **false positives** (says “probably” for an absent item) but **never false negatives**. You tune the bit-array size and hash count for your item count and target false-positive rate. (Standard Bloom filters can’t delete; counting variants can.)

39.3 Where Bloom filters are used

Guarding against **cache penetration** (Chapter 8) by rejecting keys definitely not in the database; “have I seen this URL?” in web crawlers; letting databases like Cassandra/HBase **skip disk lookups** for keys not present; malware/spam lists; and quick “is this username taken?” pre-checks.

39.4 Two more probabilistic structures

- **HyperLogLog**: estimates the **cardinality** (count of distinct items) of a huge stream — e.g. unique visitors — in kilobytes instead of gigabytes.
- **Count-Min Sketch**: estimates the **frequency** of items in a stream (e.g. how often each search term appears) in small, fixed memory.

KEY INSIGHT — APPROXIMATE CAN BE A FEATURE

When exact answers are too expensive and “close enough” is fine, a small **bounded error** buys orders-of-magnitude less memory and much faster checks. Knowing these structures turns some “impossible at this scale” problems into easy ones.

Common mistakes

WATCH OUT

- Treating a Bloom-filter “probably present” as certain (it can be a false positive).
- Under-sizing the filter and getting a high false-positive rate.
- Trying to delete from a standard Bloom filter (use a counting variant).
- Using an exact structure where an approximate one would save massive memory.

Interview questions

1. How does a Bloom filter test membership? Why no false negatives?
2. How do you tune its false-positive rate?
3. Give three real uses of Bloom filters.
4. What do HyperLogLog and Count-Min Sketch estimate?

Hands-on exercises

1. Walk an add and a check through an 8-bit Bloom filter with 3 hashes.
2. Explain why a “definitely not present” result is always trustworthy.
3. Pick a structure for: cache-miss guard, unique daily visitors, top search terms.
4. Estimate the memory saved vs an exact set for a billion items.

Mini project

GUARD A CACHE WITH A BLOOM FILTER

Build a Bloom filter over your database keys and place it in front of the cache/DB: on a lookup, if the filter says “definitely not present,” skip the database entirely. Measure the reduction in useless DB queries for random missing keys (cache penetration, Chapter 8), and observe the false-positive rate as you vary the filter size.

| Large project (capstone continues)

YOUR SOCIAL PLATFORM — PROBABILISTIC WINS

Add a **Bloom filter** to reject lookups for definitely-nonexistent users/posts before they hit the database (penetration guard). Use **HyperLogLog** to count unique viewers of posts/feeds cheaply, and consider **Count-Min Sketch** for trending-term frequencies. Note the accuracy/memory trade you’re accepting for each.

| Chapter summary

- **Probabilistic structures** give approximate answers in tiny memory, accepting a bounded error.
- A **Bloom filter** tests membership with a bit array + hashes: “**definitely not**” or “**probably**” — false positives, never false negatives.
- Uses: cache-penetration guards, crawler dedup, skipping disk lookups.
- **HyperLogLog** estimates cardinality; **Count-Min Sketch** estimates frequencies.

CHAPTER 40

Graceful Degradation & Chaos Engineering

Part VI · Reliability & Resilience — *degrade, don't collapse; and test failure on purpose.*

LEARNING OBJECTIVES

- Explain graceful degradation and give concrete examples.
- Explain load shedding and prioritizing critical work.
- Combine the Part VI patterns into defense in depth.
- Explain chaos engineering and why it builds confidence.

REAL-WORLD ANALOGY — A SHOP IN A POWER CUT

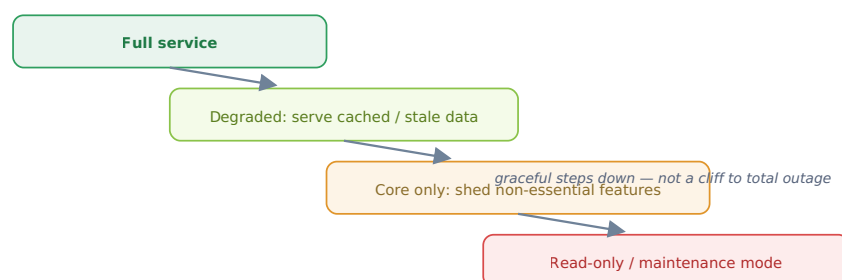
When the power fails, a well-run shop doesn't close — it switches to cash-only, candle-lit mode and keeps serving customers at reduced capability. And it holds fire drills so staff know exactly what to do *before* a real emergency. Resilient systems do both: **degrade gracefully** instead of collapsing, and **rehearse failure** deliberately.

40.1 The problem

Despite every pattern in Part VI, failures still happen. Two goals close the part: the system should **degrade gracefully** — offering reduced but useful service rather than a hard error — and its resilience should be **proven by testing failure on purpose**, not assumed.

40.2 Graceful degradation

When a component fails, provide a **reduced but useful** experience instead of an error page. Serve **cached/stale** data when the database is down (Chapter 8); disable non-essential features (recommendations, trending) to keep the core (feed, checkout) working; return a sensible **default** when a personalization service fails (fallbacks from circuit breakers, Chapter 37).



40.3 Load shedding

Under overload, deliberately **drop or reject low-priority work** to protect the core (Chapters 5, 35). Prioritize critical requests (checkout over recommendations). It's better to

serve 90% of traffic well than to let 100% degrade into failure — a controlled sacrifice beats a total collapse.

40.4 Defense in depth

KEY INSIGHT — THE PART VI PATTERNS WORK TOGETHER

No single pattern makes a system resilient; they layer. **Redundancy** (Ch34) removes single points of failure; **rate limiting** (Ch35) and **load shedding** prevent overload; **timeouts, retries, and backoff** (Ch36) bound and recover from transient faults; **circuit breakers and bulkheads** (Ch37) stop and contain cascades; **Sagas** (Ch38) keep data consistent; and **graceful degradation** keeps users served when parts fail. Reliability is defense in depth.

40.5 Chaos engineering

You don't truly know a system is resilient until you make it fail. **Chaos engineering** deliberately injects failures — killing instances, adding latency, dropping network — to find weaknesses before they cause real outages. Netflix's **Chaos Monkey** randomly terminates production instances so engineers *must* build services that tolerate it. The method: form a hypothesis ("the system stays up if a node dies"), limit the **blast radius**, run the experiment, and fix what breaks.

KEY INSIGHT — BREAK THINGS ON PURPOSE TO BUILD CONFIDENCE

Designing *for* failure (Chapter 19) is only half the job; the other half is **verifying** it under controlled, real failures. Chaos engineering turns "we think it's resilient" into "we've seen it survive."

Common mistakes

WATCH OUT

- Hard-failing the whole page when one non-essential component is down.
- No load shedding, so overload degrades *everything* instead of protecting the core.
- Relying on one pattern instead of layered defense in depth.
- Assuming resilience without ever testing real failures.
- Running chaos experiments with no blast-radius limit or rollback plan.

Interview questions

1. What is graceful degradation? Give two concrete examples.
2. What is load shedding, and how do you prioritize?
3. How do the Part VI patterns combine into defense in depth?
4. What is chaos engineering, and why do it in (or near) production?
5. How do you run a chaos experiment safely?

Hands-on exercises

1. Design fallbacks for three features so a DB outage degrades rather than fails.
2. Define a load-shedding priority order for an e-commerce site under overload.
3. Map each Part VI pattern to the failure it addresses.
4. Write a chaos experiment (hypothesis, blast radius, steps) for a single service.

Mini project

DEGRADE, THEN BREAK ON PURPOSE

Add graceful degradation to an app: when the database is unavailable, serve cached/stale data and disable non-essential features while keeping the core path working. Then run a small chaos experiment — kill the database or inject latency — with a limited blast radius, and verify the app degrades as designed instead of collapsing.

Large project (capstone continues)

YOUR SOCIAL PLATFORM — THE RESILIENCE PLAN

Write your platform's resilience plan: which features degrade (feed from cache, recommendations off) versus which must stay strongly consistent (auth, payments); a load-shedding priority order for peak overload; and a chaos experiment schedule (kill an app instance, fail a cache, add DB latency) with blast-radius limits. This plan ties together every pattern from Chapters 34–40.

Chapter summary

- **Graceful degradation** offers reduced-but-useful service (cached data, disabled extras, defaults) instead of a hard failure.
- **Load shedding** drops low-priority work under overload to protect the core.
- Reliability is **defense in depth** — redundancy, limits, timeouts/retries, breakers/bulkheads, sagas, and fallbacks layered together.
- **Chaos engineering** injects real, controlled failures to prove resilience — break things on purpose to build confidence.

COMING NEXT — PART VII

Chapters 41–44 — Security. With reliability in hand, Part VII protects the system: authentication and sessions, authorization, encryption and TLS, and defending against common attacks.